

Implementation steps

1. Read inputs: n , d , and the PA file (list of permutations, each line has n integers).
2. Precompute the exact identity ball once:
 - Set radius $r = d-1$.
 - Generate all SYT shapes with first row length $\geq n-r$.
 - Enumerate all (P,Q) standard Young tableau pairs for each shape.
 - Apply inverse RSK to each (P,Q) to obtain all permutations in $B(id, r)$.
 - Store this list as `ball_id`.
3. Repeat for many trials (random restart loop):
 - Randomly choose one permutation π_i from the current PA as the "center".
4. Temporarily remove the chosen center:
 - Build `PA_minus = PA \ {pi}`.
5. Translate the identity ball to the center ball:
 - For every τ in `ball_id`, compute $x = \pi_i \circ \tau$.
 - This generates the full ball $B(\pi_i, r)$.
6. Candidate filtering (distance test against the remaining PA):
 - For each translated ball element x , check Ulam distance:
 - For every σ in `PA_minus`, compute $du(x, \sigma)$.
 - Keep x as a candidate only if $du(x, \sigma) \geq d$ for all σ in `PA_minus`.
7. Pair test inside candidates:
 - Among the candidate list, search two permutations c_1, c_2 such that $du(c_1, c_2) \geq d$.
8. If such a pair exists, perform augmentation:
 - Construct a new PA:
 - `PA_new = PA_minus  $\cup$  {c1, c2}`.
 - This is "remove 1, add 2" $\rightarrow$ net improvement by $+1$.
9. Output on success:
 - Save `PA_new` to the output file (or overwrite PA file, depending on your version).
 - Save the removed center π_i into `removed_center.txt`.
 - Save the added pair $\{c_1, c_2\}$ into `added_two.txt`.
 - Print summary: removed center, add1, add2, old size, new size.
10. If no success within `max_trials`:
 - Do not change the PA.
 - Print "No augmentation found within max_trials".

